

PORTADA

Arquitectura de Software

Semana 14 — Estándares y Protocolos de Servicios Web (SOAP, WSDL, UDDI, RESTful, HATEOAS, HAL, GraphQL)

De contratos XML rígidos al grafo flexible de datos — con el caso real de Facebook, 2012 · Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



Una app tan lenta que Facebook casi la retira del App Store.

iOS, 2012 — el problema no era el celular: era cómo la app pedía los datos por REST.



TRANSICIÓN

El mismo endpoint, para pantallas completamente distintas

En 2012 Facebook rediseñó su app de iOS: pasó de una capa HTML5 empaquetada a una app nativa de verdad. El News Feed necesitaba datos muy distintos según la pantalla — la lista principal, el detalle de una publicación, el perfil de un amigo — pero el equipo solo tenía APIs REST tradicionales para servirse los.

Ese desajuste entre "lo que la pantalla necesita" y "lo que el endpoint devuelve" es el hilo conductor de toda esta sesión: cómo distintos estándares de servicios web resuelven (o no resuelven) ese problema.

CASO · FICHA TÉCNICA

Facebook, el News Feed nativo, 2012

Empresa	Facebook (hoy Meta)
Contexto	Reconstrucción de la app de iOS como aplicación nativa, tras abandonar el enfoque basado en HTML5
Equipo	Equipo del News Feed, con el ingeniero Lee Byron como uno de los creadores principales de la solución
Problema	Las APIs REST existentes obligaban a hacer muchas llamadas, o a traer campos de más, para armar cada pantalla
Solución interna	Un nuevo lenguaje de consulta de datos, usado internamente desde 2012
Nombre público	GraphQL , liberado como estándar abierto en 2015

Documentado por Facebook/Meta Engineering y en charlas públicas de Lee Byron sobre el origen de GraphQL.

CASO · EL PROBLEMA TÉCNICO

Over-fetching y under-fetching

Con APIs REST tradicionales, cada endpoint devuelve una forma fija de datos. Eso genera dos problemas opuestos, y la app de Facebook sufría ambos a la vez:

- **Over-fetching (traer de más):** un endpoint como `/usuario/123` devuelve 40 campos del usuario cuando la pantalla del News Feed solo necesita 3 — nombre, foto y último estado. Ese exceso de datos viajando por una red móvil de 2012 costaba tiempo real de carga.
- **Under-fetching (traer de menos):** para armar una sola pantalla (ej. una publicación con sus comentarios y los perfiles de quienes comentaron) había que encadenar varias llamadas REST distintas, una tras otra.

Ambos problemas — traer de más y traer de menos — son consecuencia directa de que REST tradicional fija la forma de la respuesta en el servidor, no en el cliente.

CASO · LA SOLUCIÓN

Un lenguaje para pedir exactamente lo que la pantalla necesita

El equipo de Facebook construyó un lenguaje de consulta donde **el cliente describe la forma exacta de los datos que necesita, y el servidor responde con esa forma exacta** — ni un campo de más, ni una llamada de menos. Ese lenguaje se usó internamente en la app de iOS desde 2012.

En 2015, Facebook abrió esa tecnología al público bajo el nombre GraphQL, y hoy la usan empresas como GitHub, Shopify y la propia Meta en producción, a gran escala.

El resto de la sesión recorre el camino completo de estándares que existían antes de GraphQL y el que vino después — de SOAP a GraphQL, en orden histórico.

TRANSICIÓN

Del caso puntual al mapa completo de estándares

GraphQL no apareció en el vacío: es la respuesta más reciente a un problema que la industria lleva más de veinte años intentando resolver de distintas formas. Para entender por qué GraphQL se diseñó como se diseñó, hay que conocer primero los estándares que lo precedieron.

Recorremos el mapa completo, del más antiguo y rígido al más reciente y flexible: SOAP, WSDL y UDDI primero; REST, HATEOAS y HAL después; GraphQL al cierre.

CONCEPTO

SOAP: el protocolo formal basado en XML

SOAP (Simple Object Access Protocol) es un protocolo de mensajería basado en XML, con reglas estrictas, para invocar **operaciones remotas entre sistemas heterogéneos**. Nació en 1998 (Microsoft) y se estandarizó en el W3C — es el protocolo que dominó la primera generación de servicios web empresariales, especialmente en banca, salud y gobierno.

- Cada mensaje SOAP es un documento XML válido, con una estructura obligatoria.
- Puede transportarse sobre HTTP, pero también sobre SMTP u otros protocolos.
- Incluye soporte nativo para seguridad (WS-Security) y transacciones — clave para la banca y el gobierno.

CONCEPTO

Anatomía de un mensaje SOAP

Elemento	Función
Envelope	El elemento raíz del documento XML — envuelve todo el mensaje.
Header	Opcional. Metadatos del mensaje: autenticación, transacciones, enrutamiento.
Body	Obligatorio. Contiene la operación invocada y sus parámetros, o la respuesta.
Fault	Dentro del Body, cuando algo falla — el equivalente SOAP a un código de error HTTP.

```
<soap:Envelope>
  <soap:Header>...</soap:Header>
  <soap:Body>
    <ConsultarSaldoRequest>
      <cuenta>123456</cuenta>
    </ConsultarSaldoRequest>
  </soap:Body>
</soap:Envelope>
```

CONCEPTO · CONEXIÓN CON LA SESIÓN ANTERIOR

WSDL: el esquema de SOAP, hecho contrato formal

En la sesión pasada hablamos del **esquema/contrato** de un servicio (s13 de la sesión anterior) sin entrar en su formato concreto. Para SOAP, ese formato es **WSDL (Web Services Description Language)**: un documento XML que describe formalmente qué operaciones expone el servicio, qué parámetros recibe cada una y qué tipos de datos usa.

Un cliente puede leer el WSDL y generar automáticamente el código necesario para invocar el servicio — es diseño "contract-first" llevado al extremo: el contrato es tan preciso que se vuelve código ejecutable por herramientas.

CONCEPTO

UDDI: el directorio que casi nadie usa hoy

UDDI (Universal Description, Discovery and Integration) es un estándar para publicar y descubrir servicios web — el rol de "registro" del que hablamos en la sesión anterior (proveedor, consumidor, registro), formalizado como protocolo.

Un proveedor publicaba su servicio y su WSDL en un directorio UDDI; un consumidor lo buscaba ahí antes de integrarlo. En la práctica, la adopción fue baja: la mayoría de las organizaciones terminó compartiendo los WSDL directamente (por documentación o acuerdos comerciales) en vez de usar un directorio público automatizado.

UDDI es hoy, en gran medida, un estándar histórico — se sigue mencionando en el catálogo de servicios web porque explica por qué la idea de "descubrimiento automático" no despegó como se esperaba.

CONTRA EJEMPLO

SOAP vs. REST

	SOAP	REST
Formato del mensaje	Solo XML, estructura estricta	Cualquiera — normalmente JSON
Transporte	HTTP, SMTP, otros	HTTP exclusivamente
Contrato	WSDL, obligatorio y formal	Opcional (ej. OpenAPI), no forma parte del estándar
Estilo	Orientado a operaciones (llamar una función remota)	Orientado a recursos (manipular un recurso con verbos HTTP)
Peso del mensaje	Alto — XML es verboso	Bajo — JSON es compacto
Uso típico hoy	Banca, salud, integraciones legadas de gobierno	APIs web y móviles modernas

CONCEPTO

REST: arquitectura, no protocolo

REST (Representational State Transfer) no es un protocolo ni un estándar formal: es un estilo arquitectónico, definido por Roy Fielding en su tesis doctoral de 2000, con un conjunto de restricciones (constraints) que un sistema debe cumplir para llamarse "RESTful".

Restricción	Qué exige
Cliente-servidor	Separación clara de responsabilidades entre quien pide y quien sirve los datos.
Stateless	Cada solicitud contiene toda la información necesaria; el servidor no guarda estado de sesión entre solicitudes.
Cacheable	Las respuestas deben poder marcarse como cacheables o no.
Interfaz uniforme	Todos los recursos se manipulan con el mismo conjunto de operaciones (verbos HTTP).
Sistema en capas	El cliente no necesita saber si habla directamente con el servidor o con un intermediario.

CONCEPTO

Verbos HTTP y códigos de estado, la "interfaz uniforme" de REST

Verbo HTTP	Operación sobre el recurso	Código de éxito típico
GET	Leer un recurso, sin efectos secundarios	200 OK
POST	Crear un nuevo recurso	201 Created
PUT	Reemplazar un recurso existente completo	200 OK
PATCH	Modificar parcialmente un recurso	200 OK
DELETE	Eliminar un recurso	204 No Content

Errores comunes: 400 (solicitud mal formada), 401/403 (autenticación/autorización), 404 (recurso no existe), 500 (error del servidor).

Usar estos códigos correctamente es parte de ser "RESTful" de verdad, no solo usar JSON sobre HTTP.

CONCEPTO

HATEOAS: el nivel de REST que casi nadie implementa completo

HATEOAS (Hypermedia as the Engine of Application State) es la restricción más exigente y menos implementada del **REST original de Fielding**: cada respuesta debe incluir los enlaces ("hipermedios") a las acciones posibles siguientes, de modo que el cliente descubra qué puede hacer a continuación sin tener esa lógica programada de antemano.

Ejemplo: al consultar un pedido, la respuesta no solo trae sus datos, sino enlaces como "cancelar este pedido" o "ver la factura" — pero solo si esas acciones son válidas en el estado actual del pedido.

CONCEPTO

HAL: un formato concreto para escribir HATEOAS

HAL (Hypertext Application Language) es una convención ligera para estructurar esos enlaces dentro de una respuesta **JSON** — una forma estandarizada de implementar HATEOAS en la práctica, en vez de que cada equipo invente su propio formato de enlaces.

```
{
  "id": 4521,
  "estado": "pendiente",
  "_links": {
    "self": { "href": "/pedidos/4521" },
    "cancelar": { "href": "/pedidos/4521/cancelar" },
    "factura": { "href": "/pedidos/4521/factura" }
  }
}
```

La clave `_links` es la convención central de HAL: agrupa todos los enlaces de navegación disponibles desde ese recurso.

TENSIÓN

REST "de verdad" vs. REST como se usa en la práctica

Fielding definió REST con las cinco restricciones de s13, incluyendo HATEOAS. En la práctica, la inmensa mayoría de las "APIs REST" que se usan hoy en la industria —incluyendo APIs muy conocidas— **no implementan HATEOAS**: devuelven JSON plano, sin enlaces de navegación, y el cliente ya sabe de antemano qué endpoints puede llamar.

Esto generó un debate real en la comunidad: ¿esas APIs son "REST" o solo "HTTP con JSON"? Roy Fielding mismo escribió en 2008 que una API sin hipermedios no debería llamarse RESTful. Aun así, el término se usa hoy de forma laxa para casi cualquier API JSON sobre HTTP con verbos.

La distinción importa para el diseño: adoptar HATEOAS/HAL de verdad reduce el acoplamiento cliente-servidor, pero tiene un costo real de implementación que muchos equipos deciden no pagar.

CONCEPTO · DE VUELTA AL CASO

GraphQL: un lenguaje de consultas, un solo endpoint

GraphQL es un lenguaje de consulta para APIs, y un runtime para ejecutar esas consultas contra los datos existentes, en el que el cliente especifica exactamente qué campos necesita y el servidor responde con esa forma exacta. A diferencia de REST, que expone muchos endpoints (uno por recurso), GraphQL típicamente expone **un único endpoint** que acepta distintas consultas.

- Resuelve el **over-fetching** del caso de Facebook (s05): el cliente pide solo los campos que necesita.
- Resuelve el **under-fetching**: una sola consulta puede traer datos anidados de varios "recursos" relacionados a la vez.

CONCEPTO

Una consulta GraphQL, comparada con REST

Para armar una pantalla con el nombre de un usuario, su foto y los títulos de sus 3 últimas publicaciones, REST tradicional necesitaría al menos dos llamadas encadenadas. En GraphQL es una sola consulta:

```
{
  usuario(id: 123) {
    nombre
    foto
    publicaciones(limite: 3) {
      titulo
    }
  }
}
```

La respuesta tiene exactamente esa misma forma — ni un campo de más, ni una llamada de menos: la solución exacta al problema documentado en el caso de Facebook (s05).

CONCEPTO EN DISPUTA

¿GraphQL reemplaza a REST?

Es común presentar a GraphQL como "el sucesor de REST", pero la evidencia de la industria no respalda un reemplazo total. GraphQL resuelve muy bien el problema de over/under-fetching en clientes con pantallas muy variadas (apps móviles, dashboards complejos), pero introduce sus propios costos: cacheo HTTP estándar deja de funcionar igual de simple (un solo endpoint, muchas formas de consulta), y el control de acceso por campo es más complejo de implementar que por endpoint.

En la práctica, muchas organizaciones —incluida la propia Meta— mantienen APIs REST para casos simples y GraphQL para clientes con necesidades de datos más variables: conviven, no se reemplazan.

Elegir entre REST y GraphQL es, otra vez, una decisión arquitectónica con trade-offs — no una cuestión de moda tecnológica.

INTERACCIÓN

Diagnosticquen su propia API

En equipos, revisen la API que están construyendo (o planean construir) en CodeF@ctory y respondan:

1. ¿Sus endpoints usan los verbos y códigos de estado HTTP correctamente (s14), o todo pasa por POST con código 200?
2. ¿Tienen algún caso de over-fetching o under-fetching (s05) en su diseño actual? ¿Cómo lo resolverían con REST puro vs. con GraphQL?
3. ¿Vale la pena para su proyecto implementar HATEOAS/HAL (s15-s16), o el costo no se justifica a su escala?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. **SOAP** es un protocolo formal basado en XML (Envelope/Header/Body); **WSDL** es su contrato formal; **UDDI** fue su directorio de descubrimiento, hoy prácticamente en desuso.
2. **REST** es un estilo arquitectónico (Fielding, 2000), no un protocolo: cliente-servidor, stateless, cacheable, interfaz uniforme y sistema en capas.
3. **HATEOAS** es la restricción de REST menos implementada en la práctica; **HAL** es una convención concreta para estructurar esos enlaces en JSON.
4. La mayoría de las "APIs REST" reales no implementan HATEOAS — un debate abierto sobre qué merece llamarse verdaderamente RESTful.
5. **GraphQL**, nacido en Facebook (2012) para resolver over-fetching y under-fetching en su app móvil, no reemplaza a REST: coexiste con él según el caso de uso.

CIERRE

Para la próxima semana

- Repasar la anatomía de un mensaje SOAP (s09) y las restricciones de REST (s13): son la base para diseñar una Web API real en la próxima sesión.
- Revisar si la API de su proyecto CodeF@ctory usa correctamente los verbos y códigos de estado HTTP (s14) — lo retomaremos como ejercicio.

La Semana 15 abre con Web APIs e Implementación de Servicios Web: cómo pasar de estos estándares en el papel a una API real, versionada y documentada.

